Cashfree payment migration

Gcode-Events · how paid registration worked before, what changed, and how it works now

Branch: feature/cashfree-payments | Base: main | Prepared 2026-10-01 | Status: sandbox, awaiting first real payment test

3	2 + 1	1	0
ORDS routes in play (1 existing, 2 new)	new tables + 1 view; no existing table altered	new Oracle package (6 procedures)	Razorpay objects changed; fallback still works

1. Summary

Goal. Replace Razorpay with Cashfree for paid event registration, validating in the Cashfree sandbox first and switching to production credentials later, without breaking anything that works today.

Approach. An additive layer. Cashfree gets its own tables, its own Oracle package, two new REST routes and its own frontend checkout loader. The existing Razorpay tables, package and routes are not modified, and a single build-time flag (`NEXT_PUBLIC_PAYMENT_PROVIDER`) selects the gateway; **Cashfree is now the default**.

The two design ideas that matter most

- **Every registration rule is checked before any money moves.** Previously capacity, duplicates, email verification and ticket limits were enforced only after the customer had paid, which could produce a charge without a ticket.
- **"Paid" is established only by asking Cashfree.** The browser's checkout result is never trusted; the backend re-reads the order from Cashfree and checks status and amount.

Where this stands — Database objects, config and the three ORDS routes exist and respond correctly; `ping` confirmed the sandbox keys and outbound connectivity; 23 of 23 frontend checks pass in a real browser against mocked backend replies. **Not yet exercised:** a real order created at Cashfree, a real checkout, and a real webhook delivery. That is the next test, in section 9.

2. How it worked before (Razorpay)

Paid registration was a three-part flow: create an order, let the customer pay in Razorpay's modal, then verify the signature and register. A webhook acted as a backup path. All payment logic lived in the Oracle package `GCODE_PAYMENTS_API`; the frontend only drove the checkout.

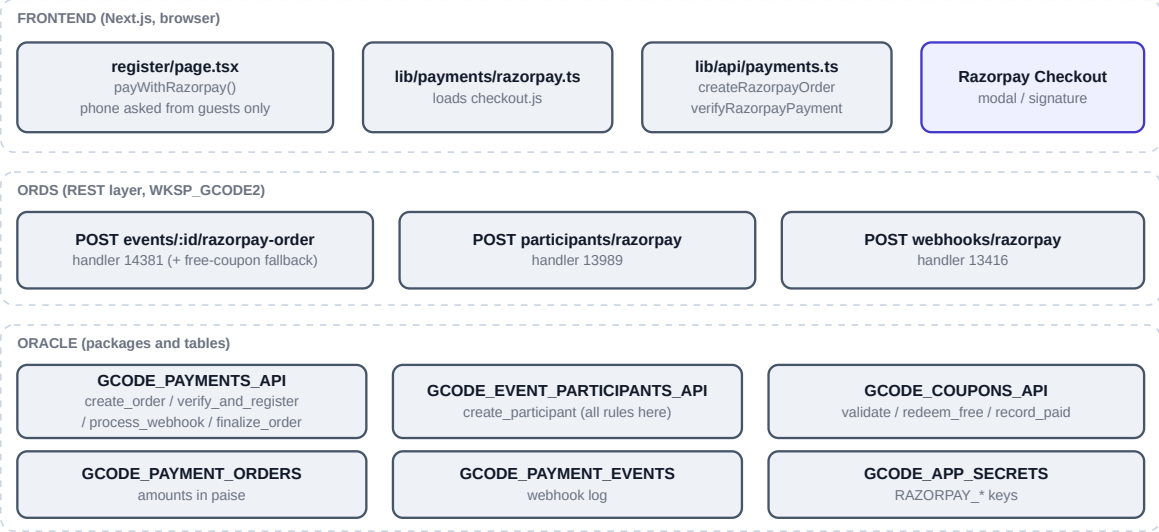


Figure 1 — Previous architecture. Everything shown is existing and remains in place.

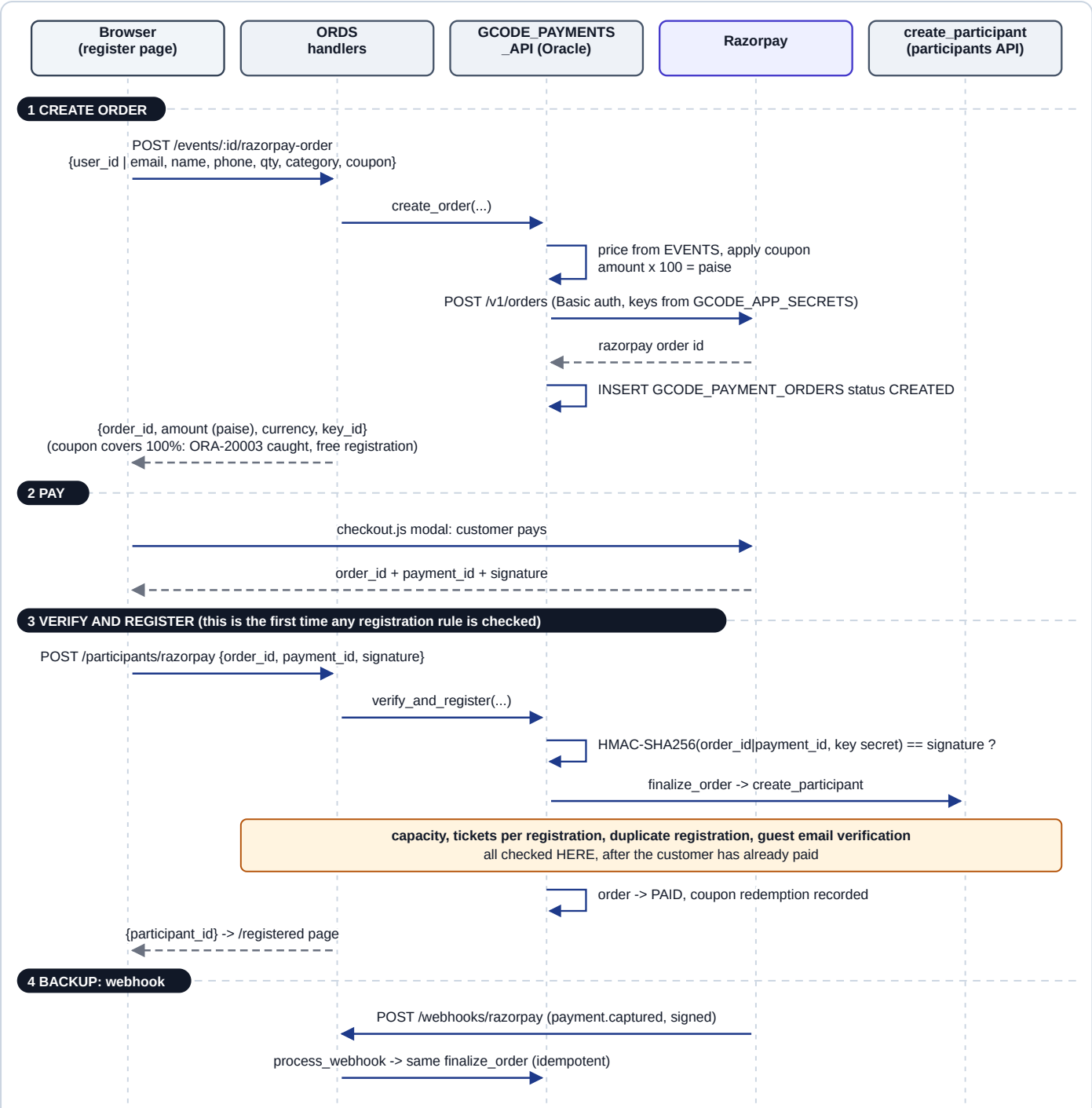


Figure 2 — Previous flow, step by step. The amber box marks the weakness this change removes: rules were only checked after payment.

What the previous system did well, and where it was weak

Good

- Server-side pricing and coupon logic; the browser never sent an amount.
- Signature verification before creating a participant.
- A webhook backup and an idempotent finalize step.
- 100% coupons skipped the gateway entirely.

Weak points removed by this change

- Capacity, duplicates, email verification and per-registration limits were checked only *after* payment, so a failure meant a charge with no ticket and no record.
- No event-status, pass-enabled or external-event check at all (a cancelled event could take payment).
- Orders never expired and there was no “paid but not registered” state.
- Webhook failures returned a blanket 400 and were not logged.

> **Note on the previous package.** The source of `GCODE_PAYMENTS_API` is in the repository (`docs/sql/coupons/06_patched_gcode_payments_api.sql`), but it was not present in the database that was inspected for this work. The new design therefore does not depend on it in any way.

3. How it works now (Cashfree)

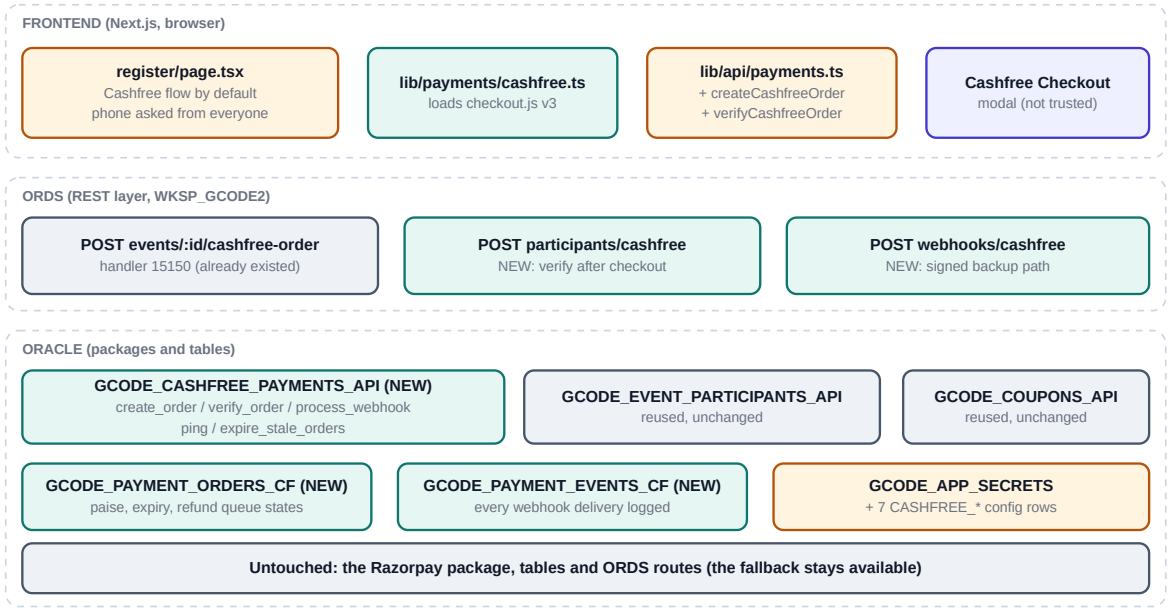


Figure 3 — New architecture. Green = new, amber = modified, grey = existing and unchanged.

3.1 Create order — everything is validated first

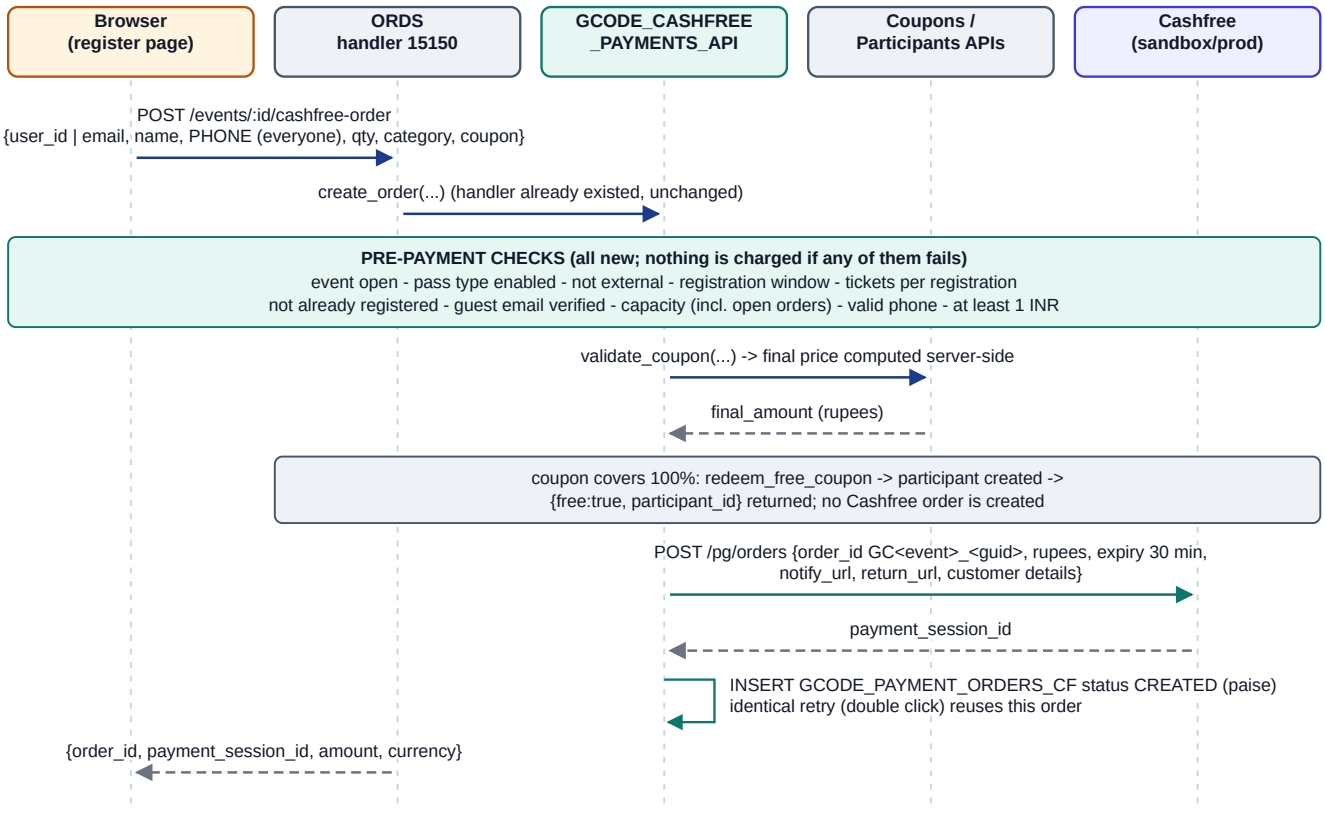


Figure 4 — Order creation. The green box lists the checks that now run before any payment is requested.

3.2 Pay and verify — the browser is never trusted

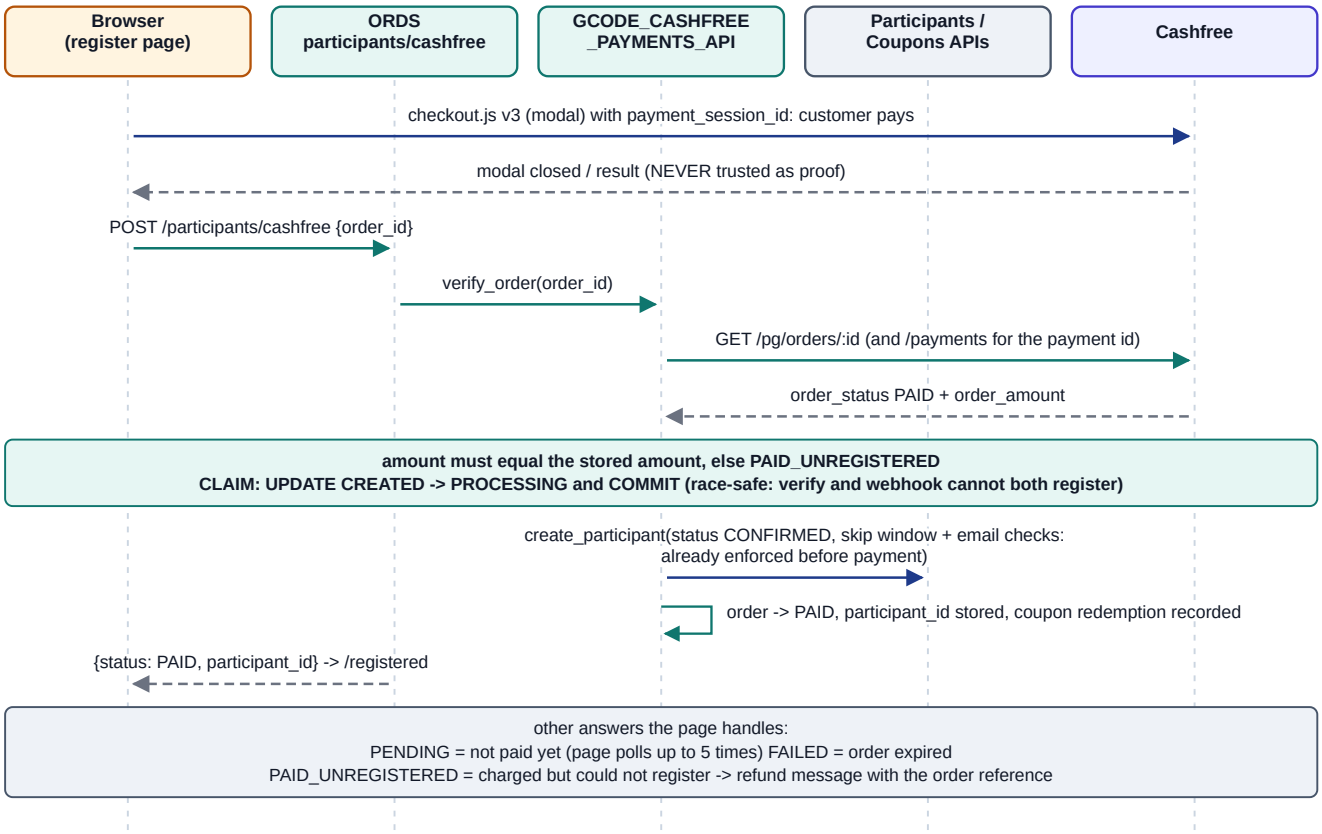


Figure 5 — Verification. The claim step prevents the browser verify and the webhook from registering the same person twice.

3.3 Backup paths — closed tab and redirect

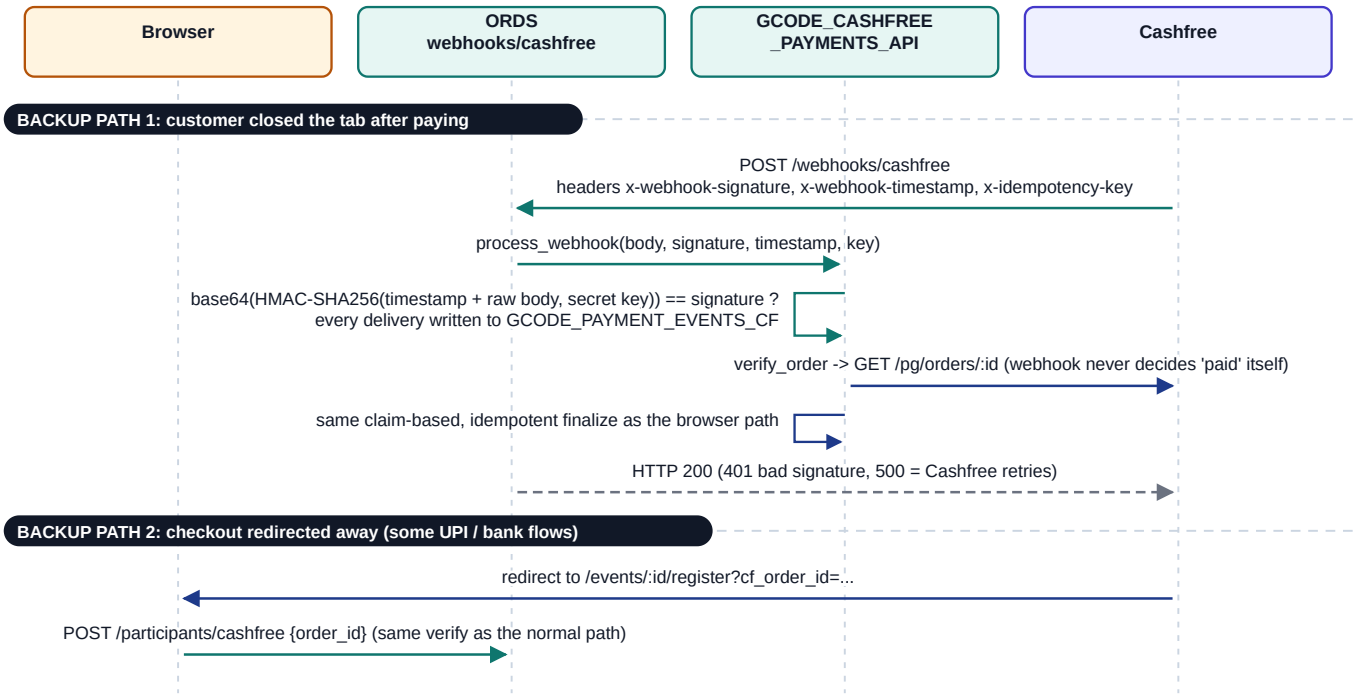


Figure 6 — The webhook and the redirect-return both funnel into the same idempotent verify.

3.4 Order lifecycle

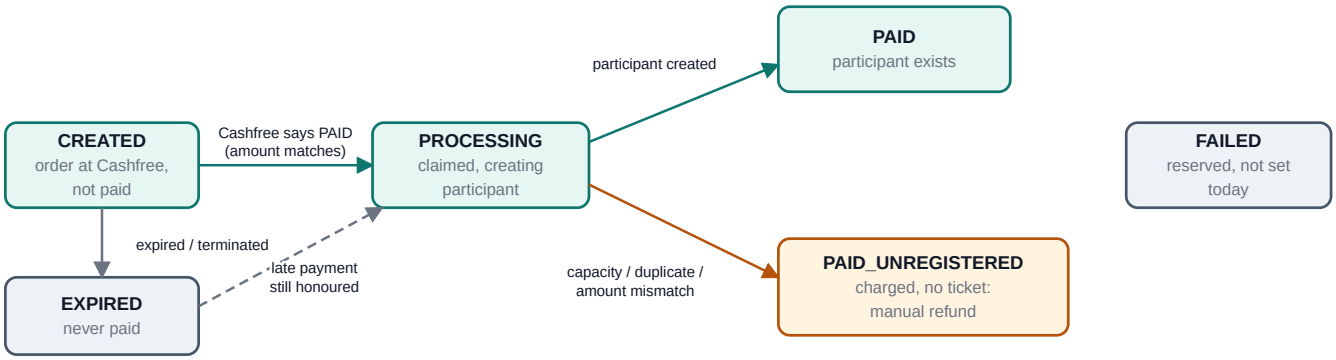


Figure 7 — Order states in GCODE_PAYMENT_ORDERS_CF. PAID_UNREGISTERED is the refund queue (view V_GCODE_PAYMENT_CF_NEEDS_ACTION).

4. Before and after at a glance

Aspect	Before (Razorpay)	Now (Cashfree)
Gateway objects	GCODE_PAYMENTS_API, GCODE_PAYMENT_ORDERS, GCODE_PAYMENT_EVENTS	GCODE_CASHFREE_PAYMENTS_API, GCODE_PAYMENT_ORDERS_CF, GCODE_PAYMENT_EVENTS_CF (Razorpay set untouched)
Proof of payment	Signature returned to the browser, verified in the database	No signature exists; the database asks Cashfree for the order status and amount
Amount unit	Paise end to end	Stored in paise (same convention); converted to rupees only for the Cashfree call
Rules checked	After payment	Before payment, and capacity/limits again at registration
Phone number	Asked from guests only	Required from every payer (Cashfree requires it), validated as 10 digits
Order lifetime	Never expires	Expires after 30 minutes (configurable); open orders reserve capacity
Charged but not registered	No record	Status PAID_UNREGISTERED plus a reason, in a queue view for manual refund
Webhook	Blanket HTTP 400, no log	401 bad signature / 500 retry / 200 ok; every delivery logged
Double click / page refresh	Could create a second order	Identical open order is reused
Frontend default	Razorpay	Cashfree; Razorpay selectable with NEXT_PUBLIC_PAYMENT_PROVIDER=razorpay

5. Registration rules: where each one is enforced

These are the rules enforced by `create_participant` (plus a few it never enforced). The key change is the second column.

Rule	Before: when checked	Now: before payment	Now: again at registration
Event open for registration (not draft / cancelled / completed)	Never	**YES**	n/a
Pass type (attendee / participant) switched on	Never	**YES**	n/a
Event is not an external-registration event	Never	**YES**	n/a
Registration window (start / deadline)	Guests only, after payment skipped	**YES** (everyone)	Skipped on purpose (customer already paid)
Tickets per registration limit	After payment	**YES**	**YES**
Capacity	After payment	**YES** (counts other people's open orders as holds)	**YES**
Already registered	After payment: silently returned the old registration	**YES** — rejected with a clear message	Detected; order flagged PAID_UNREGISTERED
Guest email verified (OTP)	After payment	**YES**	Skipped on purpose (verified earlier)
Coupon validity and price	At order creation	**YES** (unchanged logic)	Redemption recorded after payment
Phone number valid (10 digits)	Not required	**YES**	n/a
Minimum amount (1 INR)	n/a	**YES**	n/a

If a rule that cannot be fully pre-checked still fails after payment (for example the last seat is taken in the seconds between checkout and registration), the order becomes `PAID_UNREGISTERED` with a reason, and the customer sees a message with an order reference. No payment is ever silently lost.

6. What changed

6.1 Database (Oracle) — all new, run by hand in SQL Workshop

Object	What it is	Notes
`GCODE_PAYMENT_ORDERS_CF`	Cashfree orders. Every column of GCODE_PAYMENT_ORDERS (Razorpay names mapped to CF_ORDER_ID / CF_PAYMENT_ID) plus PAYMENT_SESSION_ID, USER_ID, ORIGINAL_AMOUNT, EXPIRES_ON, FAILURE_REASON, UPDATED_ON	No provider column (dedicated table). Unique CF_ORDER_ID, foreign keys to EVENTS and GCODE_COUPONS, checks on status, category, quantity, amount (at least 100 paise)
`GCODE_PAYMENT_EVENTS_CF`	Webhook audit log: CF_EVENT_ID (Cashfree's idempotency key), CF_ORDER_ID, EVENT_TYPE, PAYLOAD plus SIGNATURE_OK, PROCESSED, RESULT_NOTE	Written by an autonomous transaction, so rejected deliveries are still recorded
`V_GCODE_PAYMENT_CF_NEEDS_ACTION`	View of orders in status PAID_UNREGISTERED (rupees)	The manual refund queue
`GCODE_CASHFREE_PAYMENTS_API`	Package: create_order, verify_order, process_webhook, ping, expire_stale_orders	917 lines. Reuses GCODE_COUPONS_API and GCODE_EVENT_PARTICIPANTS_API unchanged
7 rows in `GCODE_APP_SECRETS`	CASHFREE_APP_ID, CASHFREE_SECRET_KEY, CASHFREE_BASE_URL, CASHFREE_API_VERSION, CASHFREE_NOTIFY_URL, CASHFREE_RETURN_BASE_URL, CASHFREE_ORDER_EXPIRY_MINUTES	Keys live only in the database, never in the frontend. Values are not in the repository

Package procedures. `create_order` (validates, prices, creates the Cashfree order) · `verify_order` (asks Cashfree, claims and finalizes) · `process_webhook` (signature check, then verify) · `ping` (read-only credentials and connectivity check) · `expire_stale_orders` (housekeeping).

6.2 ORDS routes

Route	Module	Status	What it does
POST /v1/events/:id/cashfree-order	gcode.events.v1	**Existed** (handler 15150), unchanged	Calls create_order. It previously failed because the package did not exist; its argument names already matched
POST /v1/participants/cashfree	gcode.participants.v1	**NEW**	Calls verify_order with {order_id}; returns {status, participant_id}
POST /webhooks/cashfree	gcode.webhooks	**NEW**	Calls process_webhook; reads x-webhook-signature / timestamp / idempotency headers; 200 / 401 / 500
razorpay-order, participants/razorpay, webhooks/razorpay	same modules	Untouched	Razorpay fallback keeps working

Neither new route needs a Parameters-grid entry: `:order_id`, `:body_text` and `:status_code` bind implicitly, as in the existing Razorpay handlers. Handler sources are in Appendix A.

6.3 Frontend

File	Change	Lines
`src/app/(public)/events/[id]/register/page.tsx`	Modified. Cashfree checkout flow (create order, open modal, confirm with backend, poll while PENDING); phone field now shown to signed-in users on paid events and validated for everyone; redirect-return handler for `?cf_order_id=`; PAID_UNREGISTERED and closed-modal messages. All new behaviour is gated, so the free-event flow is unchanged	+177 / -8
`src/lib/payments/cashfree.ts`	New. Loads Cashfree checkout.js v3 once and opens it in modal mode (sandbox/production from NEXT_PUBLIC_CASHFREE_MODE)	74 new
`src/lib/payments/provider.ts`	New. Build-time gateway switch; Cashfree is the default	10 new
`src/lib/api/payments.ts`	Added createCashfreeOrder and verifyCashfreeOrder	+50
`src/lib/api/types.ts`	Added Cashfree request / response types, including the four verify statuses	+30
`.../my-organized-events/[id]/_components/upi-claims-tab.tsx`	One sentence of organizer help text no longer mentions Razorpay	+2 / -3
docs/architecture-overview.md, component-reference.md, routing-reference.md	Updated to describe Cashfree as default and Razorpay as legacy	small

Unchanged on purpose: the Razorpay frontend code (`razorpay.ts` and the Razorpay functions) stays until Cashfree is validated, then can be deleted in a follow-up.

6.4 Documentation and scripts added

Everything under `docs/sql/cashfree/` : `01_tables.sql` , `02_gcode_cashfree_payments_api.sql` , `03_ords_endpoints.md` , `04_testing_and_rollout.md` , `99_rollback.sql` and a step-by-step `README.md` runbook.

7. Impact analysis

Area	Impact	Handling
Free events / free registration	None	Verified in the browser: same endpoint, no phone, no payment call
Razorpay flow	None (code and data)	Fallback via NEXT_PUBLIC_PAYMENT_PROVIDER=razorpay
Coupons	Price still computed server-side and frozen on the order	100% coupon takes the free path; a coupon leaving under 1 INR is rejected; last-redemption race is accepted and logged because refusing a paid customer is worse
Capacity	Now reserved at order time by open orders	30-minute expiry releases holds; checked again at registration
Participants / emails / WhatsApp	Same create_participant call and side effects	Confirmation email sent once; duplicate guard prevents repeats
Organizer revenue and '₹ paid' displays	Unchanged and still inaccurate with coupons	Pre-existing: computed in the browser as list price x quantity. Out of scope
Existing public API	No ORDS route in the workspace is privilege-protected	Pre-existing. New routes are safe because they only act on Cashfree-confirmed payments
WhatsApp package	GCODE_WHATSAPP_API body is INVALID in the database	Pre-existing, unrelated; its failure is swallowed by create_participant
Deployment order	The frontend now defaults to Cashfree	**Do not deploy the frontend until the database objects, config rows and the two ORDS routes exist in that environment**

8. Production considerations

- The database used during development is the production database; all SQL is additive and was run by hand. Nothing was written to it by automated tests (browser tests intercept every write).
- Sandbox keys are currently in the config table. Before real payments, replace them with live keys and change the base URL, notify URL, return URL and `NEXT_PUBLIC_CASHFREE_MODE` (table in section 10).
- Cashfree production requires the site domain to be whitelisted and an HTTPS webhook endpoint.

9. Verification

Check	Result	Evidence
Typecheck and lint on all changed files	**PASS**	tsc --noEmit and eslint report nothing
Package and tables compile and exist	**PASS**	Package + body VALID after fixing two compile errors found by the first run (STANDARD_HASH is SQL-only; JSON_OBJECT RETURNING CLOB is invalid in PL/SQL)
Order route reaches the package	**PASS**	POST for a free event returns ORA-20001 'Event is free' (raised by the package before any write)
Verify route	**PASS**	Unknown order returns HTTP 400 'Payment order not found'
Webhook route	**PASS**	Missing and wrong signatures both return HTTP 401; deliveries logged with signature_ok = N
Cashfree credentials and connectivity	**PASS**	ping: HTTP 404 'OK: Cashfree reached at sandbox.cashfree.com and the keys were accepted'
Frontend in a real browser (mocked backend, 23 checks)	**23 / 23**	Free regression, paid signed-in, paid guest, rejection before payment, PAID_UNREGISTERED message, closed modal, redirect return
Real order creation at Cashfree, real checkout, real webhook	**NOT YET RUN**	Needs one temporary paid event (below)

Frontend checks performed (all with writes intercepted)

- **Free event, signed in:** no phone field; calls the free endpoint; sends no phone; no gateway call.
- **Paid, signed in:** phone shown; 5 digits rejected before any request; valid phone sends user_id + phone + quantity and **no amount**; checkout opens with the backend's session id; verify polled until PAID; lands on /registered; no Razorpay call.
- **Paid, guest:** email, name and phone reach the backend; lands on /registered.
- **Backend refuses (already registered):** message shown, checkout not opened, button usable again.
- **PAID_UNREGISTERED:** refund message with the order reference. **Closed modal:** 'not completed, not charged'. **Redirect return:** verified once, lands on /registered.

Remaining test: first real payment (Cashfree sandbox)

1. Create one clearly named paid test event (small price; the database is production, so keep it brief and cancel it afterwards).
2. Register and pay with the sandbox UPI id `testsuccess@gocash`.
3. Confirm the app shows /registered, the order is PAID in `GCODE_PAYMENT_ORDERS_CF`, a CONFIRMED participant exists, and Cashfree shows the payment.
4. Confirm a webhook row with signature_ok = Y in `GCODE_PAYMENT_EVENTS_CF` (if empty, add the same URL as a dashboard endpoint — documented fallback).
5. Then run the priority cases: failed payment, closed tab after paying, already registered, bad webhook signature. The full 17-row matrix is in `04_testing_and_rollout.md`.

10. Rollout, rollback and the move to production

Rollback

- **Instant:** set `NEXT_PUBLIC_PAYMENT_PROVIDER=razorpay` and restart/redeploy.
- **Full removal:** `99_rollback.sql` drops the package, view, both tables and the config rows (export payment history first); delete the two ORDS routes.

Sandbox to production

- Run the same SQL, config rows and routes in the production environment.
- No code change; only values below.

Setting	Sandbox	Production
CASHFREE_BASE_URL (database)	https://sandbox.cashfree.com/pg	https://api.cashfree.com/pg
CASHFREE_APP_ID / CASHFREE_SECRET_KEY (database)	test keys	live keys
CASHFREE_NOTIFY_URL (database)	development webhook URL	production webhook URL, HTTPS only
CASHFREE_RETURN_BASE_URL (database)	http://localhost:3000	the real site URL
NEXT_PUBLIC_CASHFREE_MODE (frontend env)	sandbox	production
Cashfree dashboard	none required	whitelist the site domain; configure the production webhook if the per-order URL is not delivering

11. Known limitations and follow-ups

- **No automatic refunds.** Paid-but-unregistered orders are recorded and queued; refunds are done manually in the Cashfree dashboard. An automatic refund call is a possible follow-up.
- **Cashfree API shapes** follow Cashfree's documentation and were exercised only by `ping` so far. The order-creation response and the webhook payload fields are the first things to watch in the real test.
- **Unconfirmed in Cashfree's docs:** whether a per-order notify URL alone triggers webhooks or a dashboard endpoint is also needed (fallback documented).
- **Organizer revenue / '₹ paid'** ignore coupons (pre-existing).
- **Razorpay frontend code** to be removed after validation.
- The FAILED order status is allowed by the table but not set by the code today (expired orders use EXPIRED).

Appendix A — New ORDS handler sources

POST /v1/participants/cashfree (module gcode.participants.v1)

```
DECLARE
  v_status         VARCHAR2(30);
  v_participant_id NUMBER;
BEGIN
  GCODE_CASHFREE_PAYMENTS_API.verify_order(
    p_order_id       ⇒ :order_id,
    p_status         ⇒ v_status,
    p_participant_id ⇒ v_participant_id
  );
  APEX_JSON.open_object;
  APEX_JSON.write('status', v_status);
  APEX_JSON.write('participant_id', v_participant_id);
  APEX_JSON.close_object;
  :status_code := 200;
EXCEPTION
  WHEN OTHERS THEN
    APEX_JSON.open_object;
    APEX_JSON.write('error', SQLERRM);
    APEX_JSON.close_object;
    :status_code := 400;
END;
```

POST /webhooks/cashfree (module gcode.webhooks)

```
BEGIN
  GCODE_CASHFREE_PAYMENTS_API.process_webhook(
    p_payload         ⇒ :body_text,
    p_signature       ⇒ OWA_UTIL.get_cgi_env('HTTP_X_WEBHOOK_SIGNATURE'),
    p_timestamp       ⇒ OWA_UTIL.get_cgi_env('HTTP_X_WEBHOOK_TIMESTAMP'),
    p_idempotency_key ⇒ OWA_UTIL.get_cgi_env('HTTP_X_IDEMPOTENCY_KEY')
  );
  :status_code := 200;
EXCEPTION
  WHEN OTHERS THEN
    :status_code := CASE WHEN SQLCODE = -20020 THEN 401 ELSE 500 END;
END;
```

Appendix B — Error codes raised by the package

Code	Meaning (shown to the customer as plain text)
-20001	Quantity invalid; event is free; per-registration limit; not enough tickets; order not found; amount mismatch
-20002	Invalid coupon (with the coupon error code)
-20003	Email not verified (guest)
-20004	Registration has not opened / is closed
-20010 / -20011	Phone invalid / amount below 1 INR
-20012	Account not found / name and email required
-20020	Webhook rejected (bad or missing signature, oversized body)
-20030 / -20031	Missing config row / Cashfree call failed (message from Cashfree, never the keys)
-20040	Event not found / not open / pass type off / external event
-20041	Already registered for this event

Appendix C — Files in this change

Path	Type
docs/sql/cashfree/01_tables.sql, 02_gcode_cashfree_payments_api.sql, 03_ords_endpoints.md, 04_testing_and_rollout.md, 99_rollback.sql, README.md	new (SQL run by hand; docs)
src/lib/payments/cashfree.ts, src/lib/payments/provider.ts	new (frontend)
src/app/(public)/events/[id]/register/page.tsx, src/lib/api/payments.ts, src/lib/api/types.ts	modified (frontend)
src/app/(app)/(events)/my-organized-events/[id]/_components/upi-claims-tab.tsx	modified (help text)
docs/architecture-overview.md, docs/component-reference.md, docs/routing-reference.md	modified (docs)
docs/cashfree-payment-migration.pdf	new (this report)